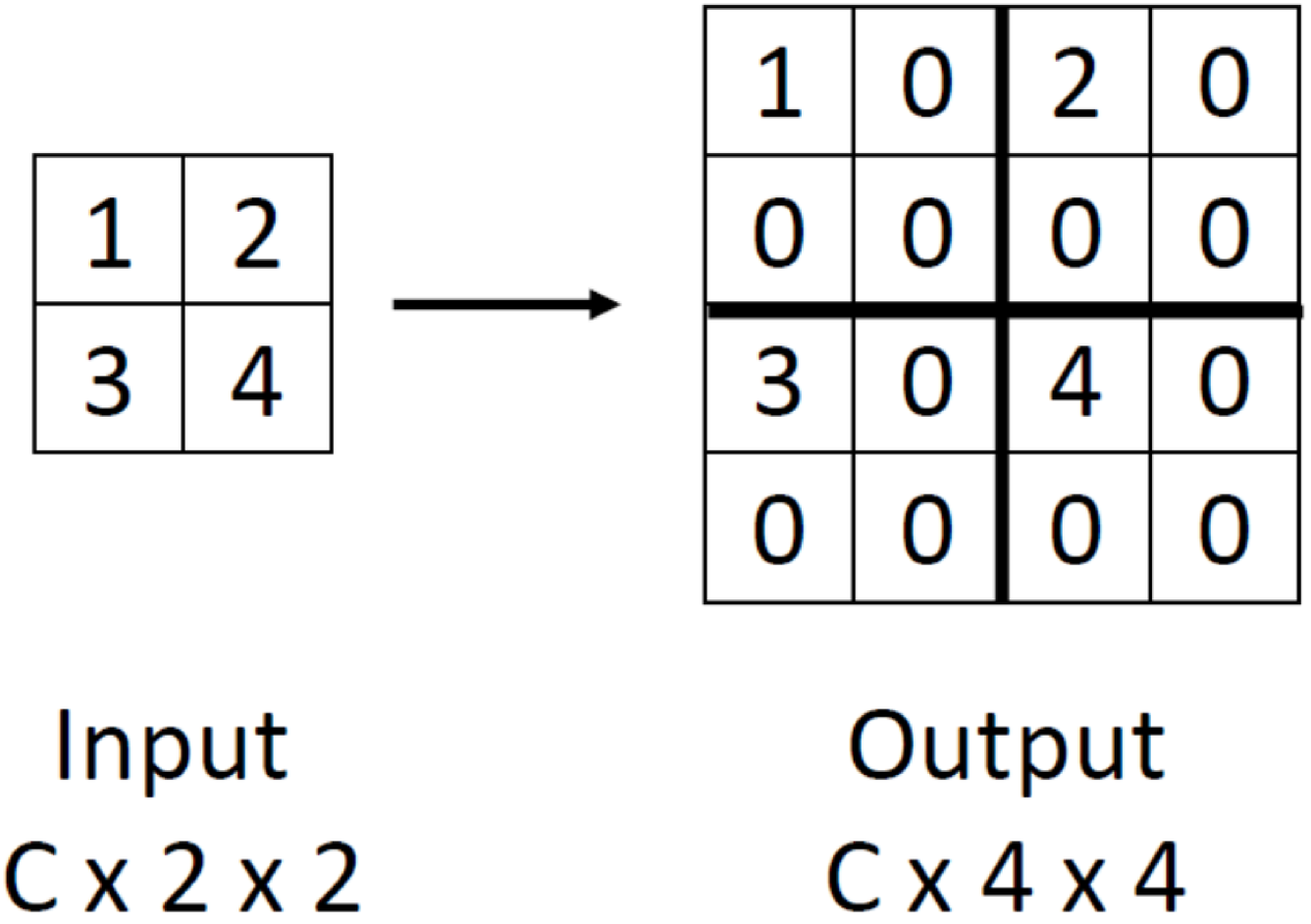
7.1 Upsampling

- the CNN layers we have seen so far, such as convolutional layers (Section 3.2) and pooling layers (Section 3.5), typically reduce (downsample) the spatial dimensions (height and width) of the input, or keep them unchanged
- in semantic segmentation that classifies at pixel-level, it will be convenient if the spatial dimensions of the input and output are the same
- for example, the channel dimension at one output pixel can hold the classification results for the input pixel at the same spatial position
- to achieve this, especially after the spatial dimensions are reduced by CNN layers, we can use other types of CNN layers that can increase (upsample) the spatial dimensions of intermediate feature maps

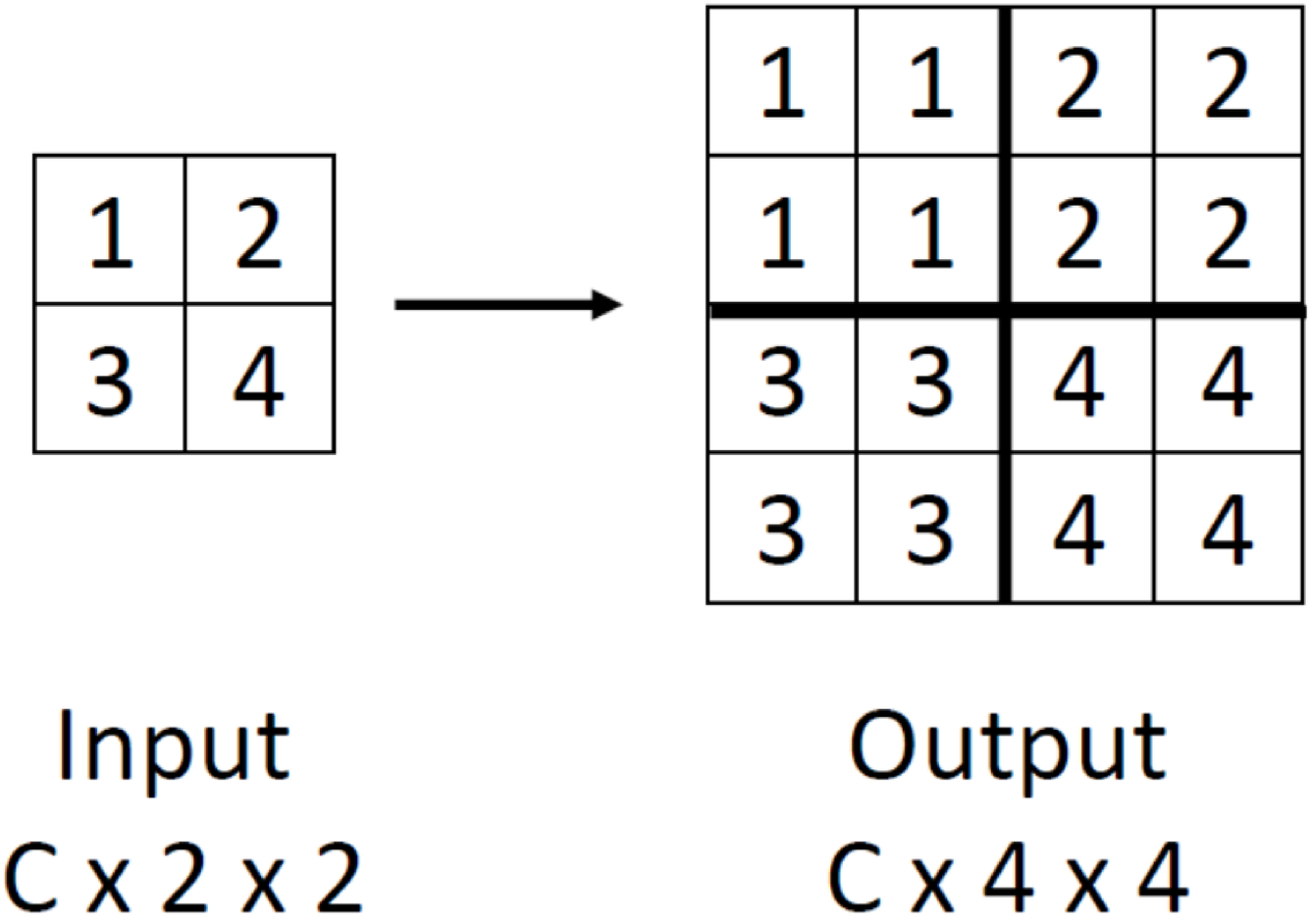
7.1 Upsampling

- *unpooling – bed of nails*



7.1 Upsampling

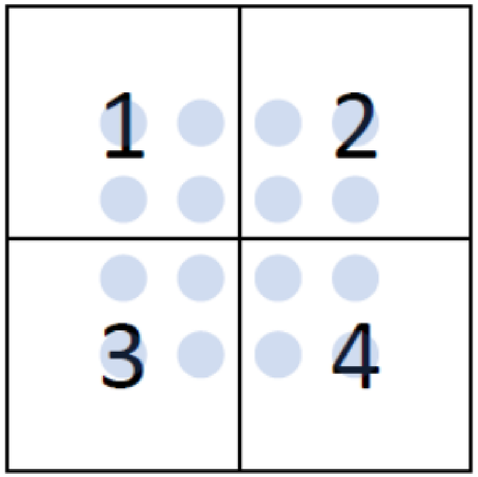
- *unpooling – nearest neighbor*



7.1 Upsampling

- *bilinear interpolation* uses four closest neighbors of (x, y) to construct linear approximations

$$f_{x,y} = \sum_{\substack{i\in\{\lfloor x\rfloor-1,\ldots,\lceil x\rceil+1\}\\ j\in\{\lfloor y\rfloor-1,\ldots,\lceil y\rceil+1\}}} f_{i,j}\max(0,1-|x-i|)\max(0,1-|y-j|)$$





1.00	1.25	1.75	2.00
1.50	1.75	2.25	2.50
2.50	2.75	3.25	3.50
3.00	3.25	3.75	4.00

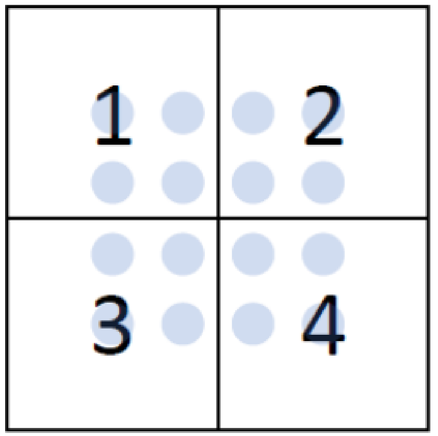
Input: C x 2 x 2

Output: C x 4 x 4

7.1 Upsampling

- *bicubic interpolation* uses sixteen closest neighbors of (x, y) to construct cubic approximations

$$f_{x,y} = \sum_{\substack{i\in\{\lfloor x\rfloor-3,\ldots,\lceil x\rceil+3\}\\ j\in\{\lfloor y\rfloor-3,\ldots,\lceil y\rceil+3\}}} f_{i,j} \max(0, 3-|x-i|) \max(0, 3-|y-j|)$$



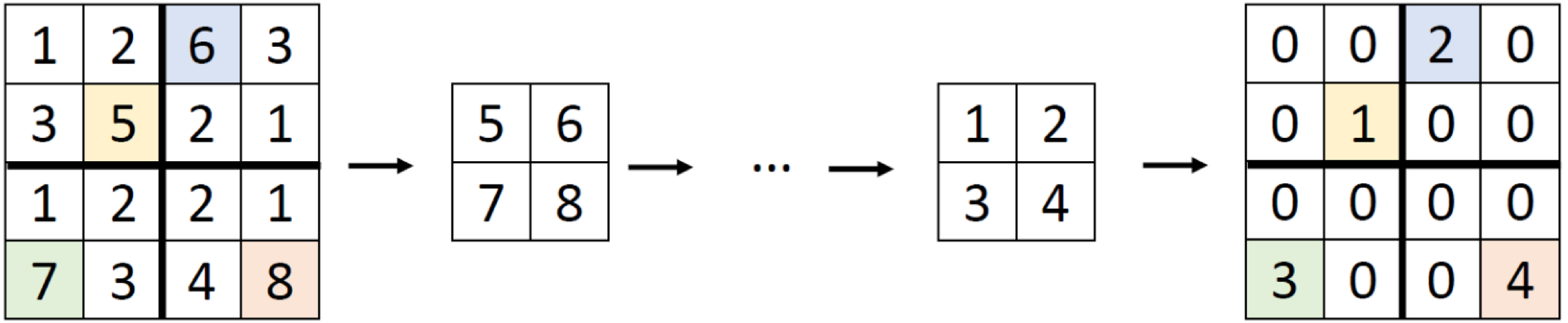
0.68	1.02	1.56	1.89
1.35	1.68	2.23	2.56
2.44	2.77	3.32	3.65
3.11	3.44	3.98	4.32

Input: C x 2 x 2

Output: C x 4 x 4

7.1 Upsampling

- for *max-unpooling*, we remember which position gave the maximum in max-pooling, and at max-unpooling we place the values into the remembered positions
- each downsampling layer is paired with an upsampling layer



7.1 Upsampling

- now, we will introduce *transposed convolution*, which is also called *fractionally-strided convolution*, for reversing downsampling operations by the convolution
- ignoring channels for now, let us begin with the basic transposed convolution operation with stride of 1 and no padding
- suppose that we are given a $n_h \times n_w$ input tensor and a $k_h \times k_w$ kernel
- sliding the kernel window with stride of 1 for n_w times in each row and n_h times in each column, yields a total of $n_h n_w$ intermediate results
- each intermediate result is a $(n_h + k_h - 1) \times (n_w + k_w - 1)$ tensor that is initialized with zeros

7.1 Upsampling

- to compute each intermediate tensor, each element in the input tensor is multiplied by the kernel, so that the resulting $k_h \times k_w$ tensor replaces a portion in each intermediate tensor
- note that the position of the replaced portion in each intermediate tensor corresponds to the position of the element in the input tensor used for the computation
- in the end, all the intermediate results are summed over to produce the output
- as an example, Figure 1 illustrates how transposed convolution with a 2×2 kernel is computed for a 2×2 input tensor

7.1 Upsampling

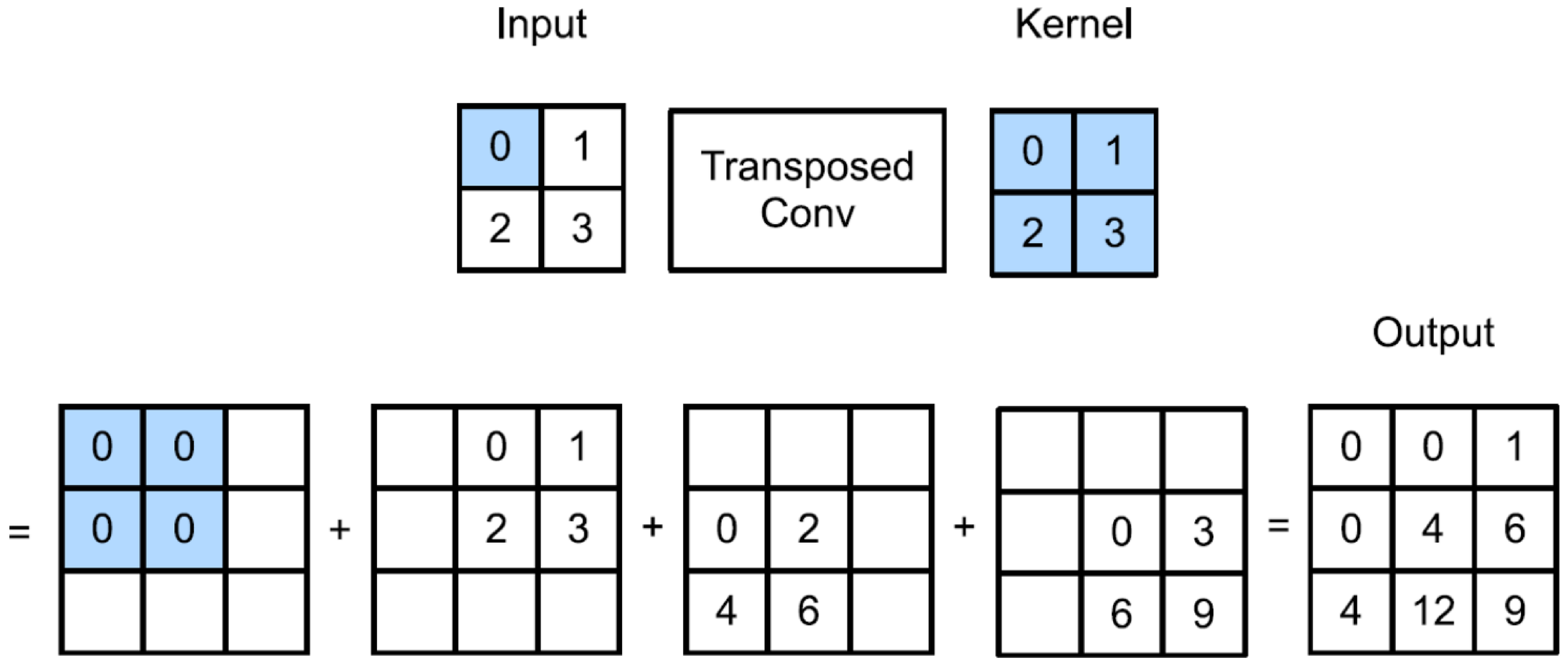


Figure 1: Transposed convolution with a 2×2 kernel. The shaded portions are a portion of an intermediate tensor, as well as the input and kernel tensor elements used for the computation.

7.1 Upsampling

- in contrast to the regular convolution (in Section 3.2) that *reduces* input elements via the kernel, the transposed convolution *broadcasts* input elements via the kernel, thereby producing an output that is larger than the input
- different from the regular convolution, where padding is applied to input, it is applied to output in the transposed convolution
- for example, when specifying the padding number on either side of the height and width as 1, the first and last rows and columns will be removed from the transposed convolution output
- in the transposed convolution, strides are specified for intermediate results (thus output), not for input
- using the same input and kernel tensors from Figure 1, changing the stride from 1 to 2 increases both the height and width of intermediate tensors, hence the output tensor in Figure 2

7.1 Upsampling

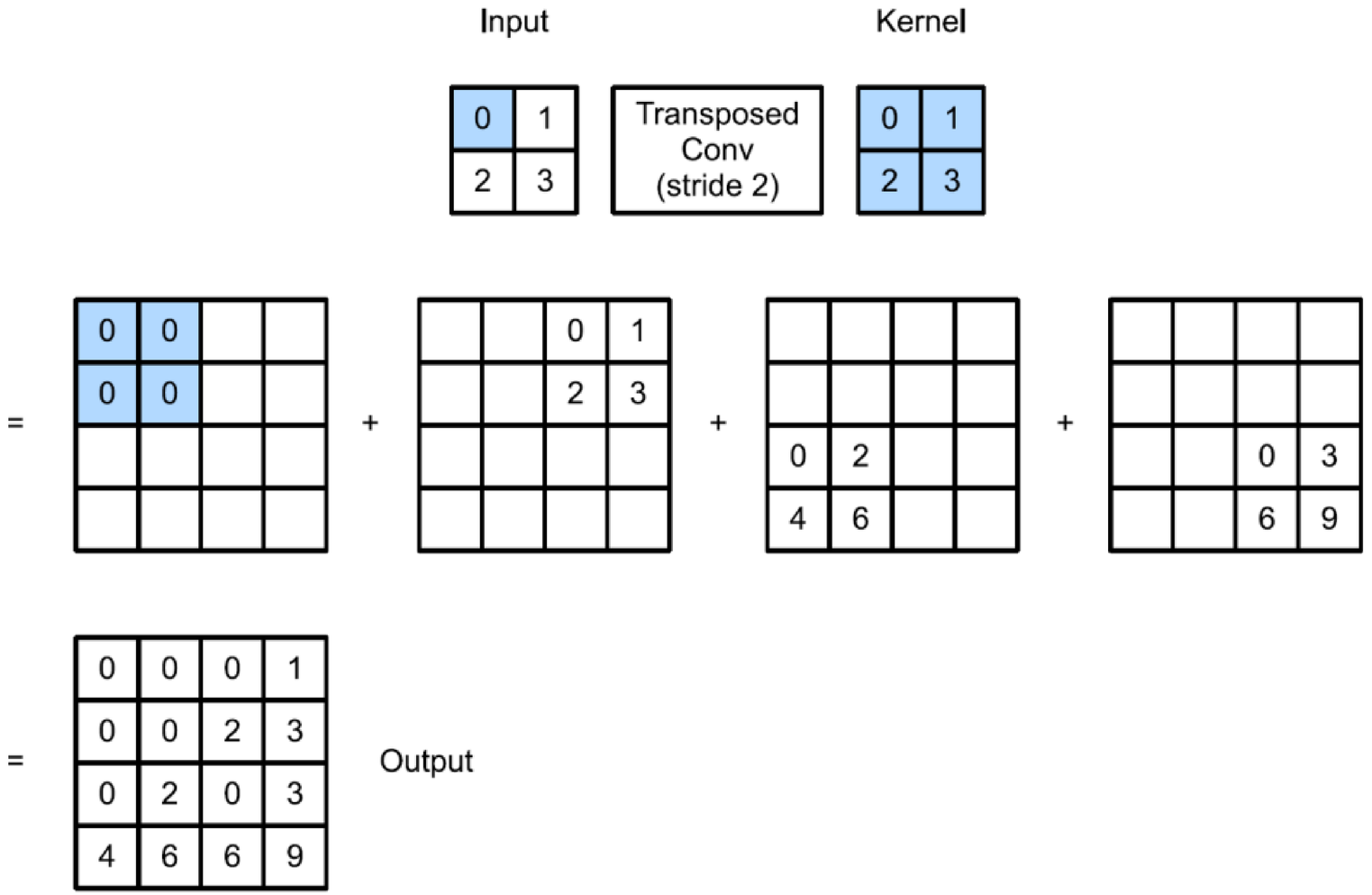


Figure 2: Transposed convolution with a 2×2 kernel with stride of 2. The shaded portions are a portion of an intermediate tensor, as well as the input and kernel tensor elements used for the computation.

7.1 Upsampling

- for multiple input and output channels, the transposed convolution works in the same way as the regular convolution
- suppose that the input has c_i channels, and that the transposed convolution assigns a $k_h \times k_w$ kernel tensor to each input channel
- when multiple output channels are specified, we will have a $c_i \times k_h \times k_w$ kernel for each output channel
- all in all, if we feed X into a convolutional layer f to output $\mathrm{Y} = f(\mathrm{X})$ and create a transposed convolutional layer g with the same hyperparameters as f, except for the number of output channels being the number of channels in X, then $g(\mathrm{Y})$ will have the same shape as X
- in general, we can see that for stride s, padding $s/2$ (assuming $s/2$ is an integer), and the height and width of the kernel $2s$, the transposed convolution will increase the height and width of the input by a factor of s